

SOLUCIÓN PRE-PARCIAL 1

PROFESOR: Federico Melo Barrero

PROGRAMACIÓN DINÁMICA — OPERACIONES MÍNIMAS, VERSIÓN A (25 DÉCIMAS)

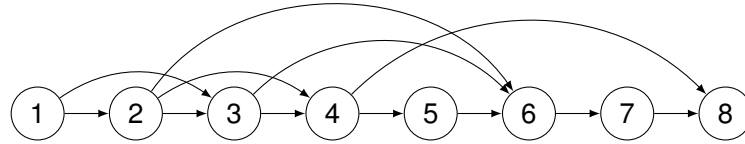
1. Sea $f(k)$ el mínimo número de operaciones para obtener k empezando desde 1, para $k \geq 1$.

$$f(k) = \begin{cases} 0 & \text{si } k = 1 \\ 1 + \min(f(k-1), [k \text{ par}] f(k/2), [k \equiv 0 \pmod{3}] f(k/3)) & \text{si } k > 1 \end{cases}$$

donde un candidato solo participa en el mín si su condición (k par, o k múltiplo de 3) se cumple. La respuesta al problema es $f(n)$.

- $f(k)$: mínimo número de operaciones para llegar a k desde 1.
- k : el número que se quiere alcanzar en este subproblema (no necesariamente el n de la entrada).
- Cada candidato del mín corresponde a deshacer la *última* operación aplicada para llegar a k : si fue “sumar 1”, antes se estaba en $k - 1$; si fue “multiplicar por 2”, antes se estaba en $k/2$ (solo posible si k es par); si fue “multiplicar por 3”, antes se estaba en $k/3$ (solo si k es múltiplo de 3). Se toma la que resulte en menos operaciones.

2. El grafo de dependencias tiene un vértice por cada $k \in \{1, \dots, n\}$, con un eje entrante desde $k - 1$ (siempre), desde $k/2$ (si k es par) y desde $k/3$ (si k es múltiplo de 3):



Dependencias entre 1 y 8: la cadena inferior es siempre $k - 1 \rightarrow k$; los arcos superiores son $k/2 \rightarrow k$ y $k/3 \rightarrow k$ cuando aplican (e.g., $3 \rightarrow 6$ y $2 \rightarrow 6$ porque $6/2 = 3$ y $6/3 = 2$).

A diferencia de una recurrencia por capas (donde k solo depende de $k - 1$), aquí $k/2$ y $k/3$ pueden estar arbitrariamente lejos de k (e.g., $k = 2^{20}$ depende de $k/2 = 2^{19}$). Por eso no basta con recordar una “capa” anterior: cualquier $f(j)$ con $j < k$ puede ser consultado por un k mucho más grande más adelante, así que hay que conservar la tabla completa. Una solución de programación dinámica requiere, entonces, $\Theta(n)$ de espacio como mínimo.

3. Tabulación bottom-up en Python, calculando f en orden creciente de k (space $\Theta(n)$, tiempo $\Theta(n)$, pues cada $f(k)$ se resuelve con trabajo $\Theta(1)$):

```

1 def min_operaciones(n: int) -> int:
2     f = [0] * (n + 1)
3     for k in range(2, n + 1):
4         f[k] = f[k - 1] + 1
5         if k % 2 == 0:
6             f[k] = min(f[k], f[k // 2] + 1)
7         if k % 3 == 0:
8             f[k] = min(f[k], f[k // 3] + 1)
9     return f[n]

```

PROGRAMACIÓN DINÁMICA — MOCHILA CON REQUISITO DE VALOR, VERSIÓN B (25 DÉCIMAS)

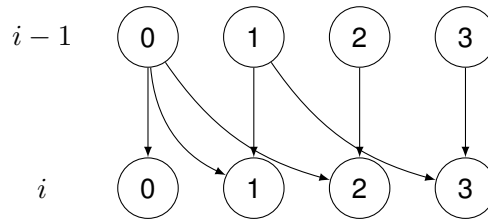
1. Sea $W(i, r)$ el peso mínimo para alcanzar, eligiendo entre los primeros i elementos, un valor de al menos r , para $i \in \{0, \dots, n\}$, $r \in \{0, \dots, V\}$. Es decir, r representa cuánto valor *falta todavía* por conseguir ($r = 0$ significa que el requisito ya se cumplió).

$$W(i, r) = \begin{cases} 0 & \text{si } r = 0 \\ \infty & \text{si } r > 0 \text{ y } i = 0 \\ \min(W(i-1, r), \text{weight}[i] + W(i-1, \max(0, r - \text{value}[i]))) & \text{si } r > 0 \text{ y } i > 0 \end{cases}$$

La respuesta al problema es $W(n, V)$.

- $W(i, r)$: peso mínimo para completar, usando entre los primeros i elementos, un valor acumulado de al menos r .
- i : cuántos de los n elementos candidatos (en el orden en que se listan) se están considerando todavía.
- r : cuánto valor falta todavía por alcanzar del requisito total V .
- El $\min$: entre no tomar el elemento i ($W(i-1, r)$, sigue faltando el mismo valor r) y tomarlo (se paga $\text{weight}[i]$ y el valor que falta baja a $\max(0, r - \text{value}[i])$, nunca por debajo de 0), se toma la que resulte en menos peso. Siempre es válido tomar el elemento i : no hay condición de “no cabe”, porque no hay límite de peso, solo se paga su costo.

2. Una tabulación de W tiene complejidad espacial $\Theta(nV)$: una entrada por cada combinación de i y r . Pero $W(i, \cdot)$ solo depende de $W(i-1, \cdot)$ (nunca de capas anteriores ni de la misma), así que el grafo de dependencias es por capas, una por cada i :



Dependencias de la capa i hacia la capa $i-1$, para $\text{value}[i] = 2$ y $V = 3$: cada (i, r) depende de $(i-1, r)$ (no tomar el elemento) y de $(i-1, \max(0, r - \text{value}[i]))$ (tomarlo). Nótese que $r = 1$ y $r = 2$ ambos apuntan a $(i-1, 0)$ al tomarlo, porque el valor sobrante se capta en 0.

Como cada capa solo depende de la inmediatamente anterior, no hace falta conservar la tabla completa: basta un arreglo de tamaño $V+1$ actualizado capa por capa, siempre que r se recorra en orden *decreciente* dentro de cada capa (así $W(i-1, \max(0, r - \text{value}[i]))$ no ha sido sobrescrito todavía cuando se necesita, pues $\max(0, r - \text{value}[i]) \leq r$). Esto reduce el espacio a $\Theta(V)$, sin cambiar el tiempo: cada uno de los $\Theta(nV)$ estados se resuelve con trabajo $\Theta(1)$, así que la complejidad temporal es $\Theta(nV)$.

3. Tabulación bottom-up en Python, con el arreglo de una sola capa (espacio $\Theta(V)$, tiempo $\Theta(nV)$):

```

1 def peso_minimo(weight: list[int], value: list[int], V: int) -> int:
2     n = len(weight)
3     W = [float('inf')] * (V + 1)
4     W[0] = 0
5
6     for i in range(n):
7         for r in range(V, 0, -1):
8             r_restante = max(0, r - value[i])
9             W[r] = min(W[r], weight[i] + W[r_restante])
10
11     return W[V]

```

GRAFOS — BFS, VERSIÓN A (10 DÉCIMAS)

- La matriz es simétrica y sin ceros en la diagonal: es un grafo no dirigido con aristas $\{v_1, v_2\}$, $\{v_1, v_4\}$, $\{v_2, v_3\}$, $\{v_3, v_5\}$, $\{v_4, v_5\}$, $\{v_4, v_6\}$, $\{v_5, v_6\}$.
- Listas de adyacencia (orden ascendente, el orden en que BFS las recorre): $v_1: v_2, v_4$ — $v_2: v_1, v_3$ — $v_3: v_2, v_5$ — $v_4: v_1, v_5, v_6$ — $v_5: v_3, v_4, v_6$ — $v_6: v_4, v_5$.

Ejecutando BFS desde v_3 . En el pseudocódigo usual, cada iteración del `while` le primero desencola un vértice y *después* recorre sus vecinos (encolando los nuevos y fijando su predecesor); ambas cosas ocurren en la misma iteración, pero en ese orden. Por eso, lo que hay en la cola y en los predecesores *justo al desencolar* un vértice u es lo que quedó tras procesar los vecinos del vértice desencolado *antes* que u — todavía no incluye lo que el propio u va a agregar, porque eso pasa después dentro de esa misma iteración:

Paso	Desencolado	Cola justo tras desencolar	Predecesores justo tras desencolar
1	v_3	$[]$	$v_3:-$ (fuente, nada más todavía)
2	v_2	$[v_5]$	+ $v_2:v_3$, $v_5:v_3$
3	v_5	$[v_1]$	+ $v_1:v_2$
4	v_1	$[v_4, v_6]$	+ $v_4:v_5$, $v_6:v_5$
5	v_4	$[v_6]$	(sin cambios: v_2, v_4 ya visitados)
6	v_6	$[]$	(sin cambios: v_1, v_5, v_6 ya visitados)

(La fila k registra lo que dejaron los vecinos del vértice desencolado en el paso $k - 1$; por ejemplo, la fila 2 — justo tras desencolar v_2 — ya trae $v_2:v_3$ y $v_5:v_3$ porque esos se fijaron al procesar los vecinos de v_3 en el paso 1, antes de este desencolado.)

Arreglo de predecesores final: $\text{pred} = [v_1:v_2, v_2:v_3, v_3:-, v_4:v_5, v_5:v_3, v_6:v_5]$.

GRAFOS — DFS, VERSIÓN B (10 DÉCIMAS)

- Las listas dan las aristas $\{1, 3\}$, $\{1, 5\}$, $\{2, 3\}$, $\{2, 6\}$, $\{3, 4\}$, $\{4, 5\}$, $\{5, 6\}$ (cada arista aparece dos veces, una en cada extremo, por ser no dirigido): 1: 3, 5 — 2: 3, 6 — 3: 1, 2, 4 — 4: 3, 5 — 5: 1, 4, 6 — 6: 2, 5.
- Ejecutando DFS (recursivo) desde 1, siguiendo el orden en que aparece cada vecino en su lista. A diferencia de BFS, aquí no hay un desfase entre “visitar” y “registrar”: el predecesor de un vértice se fija en el llamador *antes* de la llamada recursiva que lo visita, y esa llamada ya está en la pila en el instante mismo de la visita. Por eso cada fila de la tabla sí refleja, sin retraso, el estado justo al visitar ese vértice:

Paso	Visitado	Pila de llamadas	Predecesores (acumulado)
1	1	[1]	—
2	3	[1, 3]	3:1
3	2	[1, 3, 2]	+ 2:3
4	6	[1, 3, 2, 6]	+ 6:2
5	5	[1, 3, 2, 6, 5]	+ 5:6
6	4	[1, 3, 2, 6, 5, 4]	+ 4:5

Orden de visita: 1, 3, 2, 6, 5, 4. Arreglo de predecesores final: $\text{pred} = [1:-, 2:3, 3:1, 4:5, 5:6, 6:2]$.

PROYECTO — RECUPERAR RUTA, VERSIÓN A (15 DÉCIMAS)

La solución original calcula, para cada órbita i en orden creciente, dos cosas por posición j : $B(i, j)$ (el mejor costo con el que se *entra* a (i, j) desde fuera de la órbita, ya sea por ser la entrada del campo o por un portal) y $E(i, j)$ (el mejor costo tras moverse dentro de la órbita i hasta j). Para recuperar la ruta basta con recordar, además del costo, *de dónde* vino cada uno de esos dos valores:

- $\text{origen_externo}[i][j]$: cómo se llegó a $B(i, j)$ — "ENTRADA" si $(i, j) = (1, 1)$, o la posición (x_s, y_s) del portal que lo produjo.
- $\text{origen_interno}[i][j]$: la posición k de la órbita i desde la que se caminó (en línea recta, en uno de los dos barridos) hasta llegar a j con costo $E(i, j)$.

Ambos se llenan con el mismo trabajo $\Theta(1)$ por posición que ya hace la solución original al actualizar el mínimo acumulado de cada barrido, así que guardarlos no cambia la complejidad: sigue siendo $\Theta(nm)$ en tiempo y $\Theta(nm)$ en espacio.

Para reconstruir la ruta se parte de (n, m) y se deshacen los pasos: se camina en línea recta de $\text{origen_interno}[i][j]$ a j dentro de la órbita i , agregando cada posición intermedia, y desde ahí se salta a origen_externo — que es o bien la entrada (fin de la reconstrucción) o la posición de un portal en una órbita anterior, desde la que se repite el proceso.

```

1 def energia_minima_con_ruta(n, m, e, portales):
2     INF = float("inf")
3     A = [[INF] * m for _ in range(n)]
4     A[0][0] = 0
5
6     origen_externo = [[None] * m for _ in range(n)]
7     origen_externo[0][0] = "ENTRADA"
8     origen_interno = [[None] * m for _ in range(n)]
9
10    salientes = [[] for _ in range(n)]
11    for xs, ys, xe, ye in portales:
12        salientes[xs].append((ys, xe, ye))
13
14    for i in range(n):
15        fila_a, ei = A[i], e[i]
16        fila_dp = [INF] * m
17        interno = origen_interno[i]
18
19        mejor, mejor_k = INF, None
20        for j in range(m):
21            candidato = fila_a[j] - j * ei
22            if candidato < mejor:
23                mejor, mejor_k = candidato, j
24            fila_dp[j] = mejor + j * ei
25            interno[j] = mejor_k
26
27        mejor, mejor_k = INF, None
28        for j in range(m - 1, -1, -1):
29            candidato = fila_a[j] + j * ei
30            if candidato < mejor:
31                mejor, mejor_k = candidato, j
32            candidato = mejor - j * ei
33            if candidato < fila_dp[j]:
34                fila_dp[j], interno[j] = candidato, mejor_k
35
36        for ys, xe, ye in salientes[i]:
37            if fila_dp[ys] < A[xe][ye]:
38                A[xe][ye] = fila_dp[ys]
39                origen_externo[xe][ye] = (i, ys)
40
41    if fila_dp[m - 1] == INF:
42        return INF, None
43
44    ruta = []
45    i, j = n - 1, m - 1
46    while True:
47        k = origen_interno[i][j]
48        paso = 1 if j >= k else -1
49        for pos in range(j, k - paso, -paso):
50            ruta.append((i, pos))
51        destino_previo = origen_externo[i][k]
52        if destino_previo == "ENTRADA":
53            break
54        i, j = destino_previo
55
56    ruta.reverse()
57    return fila_dp[m - 1], ruta

```

PROYECTO — PORTALES CON COSTO, VERSIÓN B (15 DÉCIMAS)

1. Solo cambia $B(i, j)$, que ahora debe sumar el costo c del portal usado; $E(i, j)$ queda igual porque nunca miró el costo de *cómo* se llegó a la órbita, solo el resultado $B(i, k)$:

$$B(i, j) = \begin{cases} 0 & \text{si } (i, j) = (1, 1) \\ \min_{(x_s, y_s, i, j, c) \in \text{portales}} (E(x_s, y_s) + c) & \text{si hay portales que llegan a } (i, j) \\ \infty & \text{en otro caso} \end{cases}$$

$$E(i, j) = \min_{1 \leq k \leq m} (B(i, k) + e_i \cdot |j - k|)$$

2. No cambia: $\Theta(nm)$ se mantiene. Sumar c al construir $B(i, j)$ es trabajo $\Theta(1)$ adicional por portal, lo mismo que ya costaba consultar $E(x_s, y_s)$; los p portales siguen aportando $\Theta(p) \subseteq \Theta(nm)$, y los dos barridos por órbita ($\Theta(m)$ cada una) no se tocan.